

Juan Manuel Fraga Sastrías · ORCID orcid.org/0000-0002-9255-278X · www.docfraga.com/whitepapers/gx10-17-dias · Licencia CC BY 4.0

Infraestructura de IA 22 de julio de 2026 · 17 min de lectura

De Amazon a producción en 17 días

Compré una workstation de IA de 128 GB y en 17 días pasó de la caja a servir un modelo de 35 mil millones de parámetros en producción para mis agentes. Aquí van los números reales, la configuración exacta del motor de inferencia y los tropiezos técnicos que nadie documenta —lo justo para que tú decidas y para que tu asesor técnico verifique que está bien hecho.

Mis asistentes conversacionales —los que atienden por WhatsApp— corrían la inferencia en una Mac. Funcionaba, hasta que dejó de funcionar: al sumar agentes, un solo equipo se saturaba y la latencia se disparaba en las horas pico. Necesitaba cómputo de IA **dedicado y local**, sin nube. No por moda: por **privacidad de los datos** (información de salud que no debe salir de mi red), por costo recurrente (dejar de pagar por token) y por control (elegir el modelo y afinarlo yo). Este es el registro honesto de cómo lo resolví, con lo bueno y lo incómodo.

La máquina

Una **ASUS Ascent GX10**, del tipo DGX Spark: un superchip **NVIDIA GB10 Grace Blackwell** con **128 GB de memoria unificada** (unos 121 GB usables por modelos),

20 núcleos ARM y sistema operativo de la familia Ubuntu. Precio: alrededor de **\$88,000 MXN** —lo que cuesta una laptop premium—.

El detalle que un técnico querrá saber: la GB10 es tan nueva que reporta **capacidad de cómputo sm_121**, y eso obliga a la rama **CUDA 13** (con PyTorch compilado contra ella). Su límite conocido no es la potencia de cálculo sino el **ancho de banda de memoria (~270 GB/s)**: es lo que fija el techo de velocidad al generar texto. Por eso brilla en **inferencia sostenida multiusuario** —muchas conversaciones a la vez— y no pretende competir con un servidor de centro de datos en ráfagas extremas. Para atender agentes localmente, es exactamente la curva que quería.

Los 17 días

- **Día 0.** Compra en línea. El mismo modelo subió ~7 % de precio días después: comprar a tiempo también es ingeniería.
- **Días 1-15.** Tránsito. La caja cruzó medio país.
- **Día 16.** Llega. La probé fuera del gabinete primero: arranque sano, red privada configurada, acceso remoto por llave listo. Idle: GPU a ~37 °C y 5 W.
- **Día 17.** De madrugada —en automático, mientras dormía— quedó sirviendo su primer modelo, con reinicio automático del contenedor ante cualquier caída. Al día siguiente ya respondía a un agente en producción.

El modelo

Qwen3.6-35B: un modelo de **mezcla de expertos** (MoE) con 35 mil millones de parámetros en total pero solo **~3 mil millones activos por token** —enruta cada token a 8 de sus 256 expertos más uno compartido—. Esa es la razón de que sea rápido pese a su tamaño: calcula como un modelo chico y "sabe" como uno

grande. Es multimodal (texto, imagen y video), de pesos abiertos (Apache 2.0) y admite contextos larguísimos (262K tokens, hasta ~1M con extensión).

Un matiz que resultó decisivo: no es un MoE clásico basado solo en atención, sino de **arquitectura híbrida** —mezcla capas de atención con capas de estado tipo **Mamba / Gated DeltaNet**—. Esa elección lo hace eficiente en contextos largos, pero cambia las reglas de cómo hay que servirlo (lo pago en el tropiezo 2). Lo corro con **vLLM** en un contenedor, con pesos cuantizados a 4 bits en formato **NVFP4**, pensado para esta generación de GPU.

Cómo lo sirvo: el motor afinado

Servir un modelo no es "encenderlo": es un puñado de decisiones que separan una demo de algo que aguanta usuarios reales. Estas son las que activé en vLLM y el porqué de cada una, sin rodeos:

Decisión	Por qué
Cuantización NVFP4 (4-bit)	Formato de 4 bits de esta generación Blackwell; el proceso del modelo de 35B reserva ~34 GB (con 0.28 de utilización de memoria, KV-cache incluida) con backend MoE MARLIN NvFp4.
KV-cache en FP8	Duplica la memoria útil de contexto: ~70 GiB de caché a 40K tokens → decenas de conversaciones vivas a la vez.
Prefix caching + chunked prefill	Reaprovecha el prompt de sistema compartido entre turnos y trocea el prellenado para no bloquear a los demás usuarios.
Batch mínimo fijo (8192 tokens)	Requisito de la arquitectura híbrida Mamba: sin él, el motor ni arranca (ver tropiezo 2).
Decodificación especulativa (MTP x3)	El modelo propone varios tokens por paso y se verifican en bloque: menos latencia por respuesta.
Tool-calling + parser de razonamiento	Deja a los agentes invocar herramientas (formato XML del modelo) y separa la cadena de pensamiento de la respuesta final.

Juntas, estas piezas encienden los caminos rápidos de la GPU: el backend de MoE cuantizado, el motor de atención optimizado, la caché de contexto comprimida y la decodificación especulativa. El efecto medible es doble: **baja la latencia** de una sola respuesta y, sobre todo, **multiplica cuántas conversaciones caben en paralelo** sin degradarse.

Los números

Corrida real, con el modelo ya caliente y la configuración optimizada:

Conversaciones en paralelo	Latencia	Throughput agregado
1	0.7 s	112 tok/s
4	1.1 s	271 tok/s
8	1.3 s	493 tok/s
16	1.8 s	669 tok/s

Lo interesante no es un número, es la **forma**: la latencia casi no sube al escalar de 1 a 16 conversaciones (0.7 s → 1.8 s) mientras el throughput se multiplica ~6× (112 → 669 tokens/s). Esa es la firma de una GPU que **agrupa peticiones de verdad** —las procesa juntas en un mismo lote— y no que las hace en fila. Es justo lo que un chip Apple no puede: al llegar la segunda conversación, la serializa. Aquí conviven.

Tras afinar, una respuesta de un solo usuario bajó de **2.9 s a 0.7 s**. Durante la prueba: 95 % de uso de GPU, 57 °C y solo 28 W —menos que un foco—. Y por la matemática de la caché: con el KV en FP8, a 40K tokens de contexto quedan ~70 GiB para conversaciones vivas, techo teórico de decenas de usuarios simultáneos. En operación real reservo menos memoria a propósito, para dejarle hueco a los otros servicios (más abajo).

Los tropiezos que nadie documenta

1 • GPU tan nueva que rompe las recetas

La GB10 (capacidad sm_121) exige CUDA 13 y la imagen más reciente del servidor de inferencia. Con una versión anterior, el cargador de pesos NVFP4 revienta buscando parámetros que en ese formato ni existen, y el modelo *ni siquiera carga*. Ninguna guía lo advertía todavía: hay que exigir la imagen nueva, no la "estable" de hace unos meses.

2 • No es un MoE "normal": el peaje de Mamba

Por ser híbrido (capas de estado tipo Mamba), al activar la caché de prefijos el motor *aline*a la memoria en bloques de tamaño fijo. Si el lote máximo de tokens queda por debajo de ese bloque, el motor *ni siquiera arranca* y lanza un error de aserción críptico. La cura es fijar el lote mínimo bien alto (8192 tokens). Un solo parámetro, que no aparece en ningún tutorial y que —peor— parecía un problema de memoria cuando en realidad no lo era. Me costó horas de diagnóstico perseguir la pista equivocada.

3 • Un apagón sucio corrompe el contenedor

Tras un corte de luz no limpio, la capa de construcción del runtime de contenedores se corrompe y el servicio no levanta. La cura es apartar esa capa dañada, limpiar el estado del servicio fallido y reiniciar el runtime; los contenedores marcados para revivir vuelven solos. Lección de resiliencia real, no de laboratorio: en infraestructura propia el apagón es parte del modelo de fallas, y hay que diseñar para que la máquina regrese sola.

Tres cargas en una sola caja

La misma máquina no sirve un modelo, sirve tres cargas a la vez, cada una para un trabajo distinto:

Chat general

El Qwen3.6-35B, residente. Es el que atiende a los agentes conversacionales.

Segmentación de tomografía

Un modelo de visión 3D que delinea ~127 estructuras anatómicas en un estudio de TC en segundos. Corre bajo demanda, no residente.

Embeddings de búsqueda

El vectorizador que convierte documentos en números para la búsqueda semántica (RAG) de los agentes.

También probé una cuarta, un modelo médico de texto (gpt-oss-20b y MedGemma-27B), y la descarté: el Qwen general le ganó en mi evaluación interna, y MedGemma, por denso, no pasó de ~13 tokens/s. En producción corre Qwen3.6-35B; la máquina también se usa de vez en cuando para pruebas con otros modelos, y esas pruebas se hacen deteniendo antes el de producción (la memoria unificada es compartida; nunca en paralelo): la prueba de generación de video corrió de noche, con Qwen apagado. Las tres cargas restantes no caben todas al máximo en 121 GB, así que hay una **coreografía de memoria**: de día reservo lo justo para los agentes y dejo hueco para que la segmentación de TC entre a ráfagas; de noche apago el chat y libero la máquina para entrenar modelos y construir los índices de búsqueda. La misma caja hace turno de día (atención) y turno de noche (mejora), sin comprar una segunda.

Cómo elijo qué modelo corre: arenas propias

Los benchmarks públicos no contestan mis preguntas: no me dicen si un modelo aguanta mis casos, mis herramientas ni mi idioma. Por eso armo **arenas propias**: enfrento modelos entre sí con casos míos —casos clínicos con respuesta de referencia, cadenas de herramientas de mis agentes con una trampa deliberada,

turnos en español de México— y los califican jueces a ciegas con criterios que defino yo: éxito en la tarea, seguridad al usar herramientas, calibración y latencia del turno completo. Así se decidió qué corre en producción y qué se descarta, incluido el modelo médico de arriba.

Dos lecciones. Primera: **los tokens por segundo no predicen la experiencia**; un modelo denso con razonamiento explícito era solo 2.4× más lento por token, pero tardaba 71 s por turno contra 2.2 s. Segunda: un modelo puede ser bueno en promedio y aun así inseguro con acciones irreversibles; el de producción ejecutó un borrado que no debía en 6 de 10 repeticiones. Cambiar de modelo solo mueve la probabilidad, así que la respuesta fue de arquitectura: compuertas de confirmación en código y un clasificador de irreversibilidad que escala esos turnos a un modelo de nube. Las arenas, con sus cifras, están en el apéndice técnico.

En producción

El modelo no se conecta directo a cada agente: pasa por un **gateway propio** compatible con el estándar de la industria (el mismo contrato de API que usa la nube), así que para el agente hablar con mi GPU local o con un modelo remoto es idéntico —solo cambia un texto de ruteo—. El gateway **enruta por agente** (cada asistente puede tener su modelo) y, si el local falla, hace *fallback* automático a un modelo en la nube con un **interruptor (breaker)** que espera unos minutos antes de reintentar el local, para no golpear un servicio caído. Los agentes nunca ven el error; ven una respuesta. Esa es la diferencia entre un experimento y algo en lo que descansa el trabajo.

Apéndice técnico

Para quien tenga que reproducir o auditar este trabajo: los parámetros y las mediciones detrás del artículo. Solo figura lo que quedó registrado; lo que no se midió se declara como tal.

1 • Presupuesto de memoria en 128 GB unificados

Son 128 GB físicos, ~121 GB usables. CPU y GPU comparten el mismo pool, así que un “offload a RAM del CPU” no libera nada: el repositorio oficial de un modelo de video pesaba 123.6 GB y no cabía ni estando solo, por lo que cuantizar fue requisito, no opción.

Componente	Reserva o medición	Nota
Qwen3.6-35B NVFP4 (vLLM, producción)	~34 GB reservados (<code>--gpu-memory-utilization 0.28</code>)	KV-cache FP8 fija de 7.0 GiB; contexto de 128K.
Embeddings bge-m3	~1.6 GB	1024 dimensiones; residente.
Modelo de prueba denso de 31B (detenido el 12-sep-2026)	~30 GB reservados (25 %) · 4.7 GB en uso real	10 peticiones en 10 días, todas locales.
Estado de la máquina, 12-sep-2026	93 de 121 GB ocupados (2 libres) → 48 ocupados (73 disponibles) al detener el de prueba	Liberó ~45 GB, más que lo reservado: el proceso arrastra overhead.
Generación de video por lotes (MiniMax-H3, pesos podados de ~42.5 GB)	Pico total 92 GB con Qwen apagado · 115 de 121 GB con Qwen encendido	Clip de 15 s: 989 s vs 1300 s; clip de 5 s: 241 s vs 370 s (~35 % más lento por presión de memoria).

- **Reservar no es usar.** `--gpu-memory-utilization` aparta memoria, no refleja consumo; la planeación se hace con lo reservado. Bajar la reserva a veces basta para ganar margen sin apagar nada.
- **Un segundo vLLM compite por la misma memoria.** Levantarlo junto al de producción mató el motor del primero y lo dejó caído ~6 min. Regla: detener antes, nunca en paralelo, y verificar con una inferencia real, porque el puerto seguía respondiendo con el motor muerto.
- **Los trabajos pesados van de noche con el chat apagado.** Un script restaura el chat y lo verifica con una inferencia real antes de terminar.

2 • Metodología del benchmark

Aspecto	Registro
Fecha y estado	15-jul-2026, corrida en caliente (modelo ya cargado y sirviendo). El arranque en frío no se midió.
Ruta	Peticiones por un túnel de red privada, desde otra máquina, al endpoint compatible con OpenAI del contenedor vLLM.
Configuración	<code>--gpu-memory-utilization 0.8</code> , <code>--max-model-len 40960</code> , <code>--max-num-seqs 32</code> , KV-cache FP8, MTP ×3.
Concurrencia	1, 4, 8 y 16 conversaciones en paralelo.
Qué se midió	Latencia por respuesta (s) y throughput agregado de generación (tokens/s) sumado entre las conversaciones.
Efecto de la afinación	Un solo usuario: 2.9 s → 0.7 s. Durante la prueba: GPU al 95 %, 57 °C, 28 W.
No quedó registrado	Herramienta de carga, longitud de prompt y de salida, número de repeticiones y percentiles (p50/p95). Las cifras son puntuales, no distribuciones.

Segunda serie, 16-jul-2026, con otra configuración (`--gpu-memory-utilization 0.40`). No es comparable punto a punto con la tabla de “Los números”.

Conversaciones en paralelo	Throughput agregado
1	92 tok/s
8	313 tok/s
16	407 tok/s
32	546 tok/s

Con 32 conversaciones la latencia rondó 5 s (~6× el throughput de un solo usuario). La caché de 1.11 M de tokens da ~27 peticiones simultáneas con contexto lleno de 40K y hasta 32 con prompts normales.

3 • Configuración de vLLM

Contenedor con GPU, `--gpus all --runtime nvidia --ipc=host --restart unless-stopped`. La tabla del cuerpo explica el porqué general; esta lista los valores exactos.

Parámetro	Valor	Por qué
Imagen	<code>vllm/vllm-openai:latest</code> (≥ 0.25 ; $\sim 0.25.1$ en su momento)	Con <code>v0.18.0-cu130</code> el cargador de NVFP4 falla (<code>KeyError ... experts.w2_input_scale</code>). La GB10 es capacidad de cómputo 12.1 (sm_121) y exige CUDA 13.
Modelo	<code>nvidia/Qwen3.6-35B-A3B-NVFP4</code>	Backends activos: MARLIN NvFp4 MoE, FlashInfer y DeepGEMM.
<code>--gpu-memory-utilization</code>	0.6 $\rightarrow$ 0.8 (benchmark) $\rightarrow$ 0.40 $\rightarrow$ 0.28 (producción)	Reserva, no uso. Se bajó para convivir con otros servicios en la misma memoria.
<code>--max-model-len</code>	32768 $\rightarrow$ 40960 $\rightarrow$ 131072 (3-ago-2026)	Subido a 128K para una herramienta de programación de respaldo que fallaba al leer archivos grandes. No cuesta memoria (PagedAttention asigna por uso real); solo baja la concurrencia máxima teórica: $9.57\times$ a 40K $\rightarrow$ $4.04\times$ a 128K.
<code>--max-num-seqs</code>	4 $\rightarrow$ 32	Techo de secuencias simultáneas.
<code>--max-num-batched-tokens</code>	8192	Arquitectura híbrida Mamba: con prefix caching debe ser $\geq$ al tamaño de bloque (2144) o el motor no arranca.
<code>--kv-cache-dtype</code>	fp8	Duplica el contexto útil por GB: ~ 70 GiB de caché a 40K tokens en la

Parámetro	Valor	Por qué
		configuración del benchmark.
<code>--enable-chunked-prefill · --enable-prefix-caching</code>	activados	Prompt de sistema compartido reutilizado entre turnos; el prellenado en trozos no bloquea a los demás.
<code>--speculative-config</code>	<code>{"method":"mtp","num_speculative_tokens":3}</code>	Decodificación especulativa con las cabezas MTP del propio modelo.
<code>--tool-call-parser qwen3_xml · --enable-auto-tool-choice</code>	activados	Tool-use medido 4/5 frente a 2/5 en el servidor MLX de Apple, que emite JSON plano y resuelve parcial.
Razonamiento	<code>--reasoning-parser qwen3</code> al inicio → retirado; plantilla de chat propia con razonamiento apagado por defecto y activación opcional (<code>enable_thinking</code>)	Con el razonamiento activo por defecto el modelo gastaba todo <code>max_tokens</code> pensando y devolvía contenido vacío; el parser, además, clasificaba mal respuestas directas en el campo de razonamiento.
Embeddings	<code>--runner pooling</code> (bge-m3, 1024 dim.)	<code>--task embed</code> no existe en vLLM 0.25.1.

Probado y descartado

Modelo	Medición	Causa o requisito
MedGemma-27B (denso)	BF16 4 tok/s · FP8 dinámico 8 · int4 W4A16 Marlin + n-gram ~13; el MoE de Qwen ronda 100 tok/s.	head_dim 256/512 sin soporte de FlashInfer en sm_121 → cae a un kernel Triton lento (issues de vLLM #40677, #38887, #20865). Cuantizar acelera los GEMM de pesos, no la atención; n-gram no ayuda en generación libre.
gpt-oss-20b (MXFP4)	34–44 tok/s; en la evaluación interna perdió frente a Qwen (3/5 y 1/5 contra 5/5).	Para arrancar hizo falta <code>--enforce-eager</code> (la compilación de CUDA graphs se colgaba ~40 min) y pre-descargar los vocabularios tiktoken (el cliente Rust de harmony ignora el almacén de certificados; vLLM #22525).

4 · Fallas vistas en producción y su mitigación

Falla	Causa	Mitigación
Motor del Qwen muerto al levantar un segundo vLLM (14-ago-2026, ~6 min fuera de servicio)	Memoria unificada compartida: <code>CUDA error: an illegal memory access</code> . El valor de utilización del contenedor que ya estaba arriba no lo protege.	Detener antes, nunca en paralelo; verificar con una inferencia real.
<code>/health</code> responde 200 con el motor muerto	El endpoint no ejerce el motor.	Sondeo con generación real. Mismo patrón visto antes en el servidor MLX de Apple: el watchdog genera 5 tokens (timeout 10 s) cada 5 min y reinicia hasta dos veces antes de pedir intervención manual.
Arranque abortado con aserción de Mamba al relanzar el contenedor	Se omitió <code>--max-num-batched-tokens 8192</code> . El síntoma engañaba: parecían OOM por fragmentación.	Copiar todos los flags de fábrica al relanzar cualquier contenedor.
Fragmentación de memoria tras muchos cambios de contenedor	Vaivén de cargas y descargas en memoria unificada.	Reiniciar el equipo; los contenedores con <code>unless-stopped</code> reviven solos.

Falla	Causa	Mitigación
Corte de luz sucio: el servicio de contenedores no levanta	Base de datos de BuildKit de Docker corrupta.	Apartar la base dañada, limpiar el estado fallido del servicio y reiniciarlo. El cliente de red privada corre como servicio de arranque (sin sesión gráfica), así que la máquina vuelve sola.
Alerta «motor local inalcanzable» durante pruebas nocturnas planeadas	La alerta era legítima: el motor estaba apagado a propósito.	Toda corrida nocturna debe avisar o silenciar el monitor en su ventana.
Modelos visibles pero ilegibles dentro del contenedor	El caché de Hugging Face guarda symlinks; al montar solo un subdirectorío quedan rotos (<code>FileNotFoundError</code> aunque el listado muestre el modelo).	Árbol con hardlinks, o montar el caché completo.
Un fallback que enmascara al primario roto (jul-ago 2026)	El id llevaba prefijo de proveedor y la llave real lo rechazaba con 403. De todos los mensajes de los canales «locales» solo uno lo sirvió el modelo local; 11 eventos de fallback; con el breaker abierto no se registraba nada.	Endpoint de métricas de ruteo: configurado vs quién respondió de verdad (<code>ok</code> / <code>divergente</code> / <code>sin_datos</code>); el registro ahora incluye el motivo (<code>error_primario</code> o <code>breaker_abierto</code>). Validado el 5-sep-2026: una llave de nube dejó de ser válida (401) y el detector lo marcó.

5 • Ruteo: un gateway delante de modelos locales y de nube

Los agentes hablan con un solo endpoint compatible con OpenAI. Pasar de la GPU local a un modelo de nube es cambiar una cadena de modelo en la configuración del canal: el agente conserva su SDK y su contrato, por eso no hubo que reescribirlos.

Pieza	Detalle
Contrato	Un gateway compatible con OpenAI. Forma canónica: <code>image_url</code> al estilo OpenAI; litellm traduce al formato de Anthropic cuando el destino es nube.

Pieza	Detalle
Identidad	Una llave por agente, con lista de modelos permitidos y tope de peticiones por minuto. La lista se aplica al completar (403), no al listar modelos. El id del modelo va sin prefijo de proveedor.
Primario y respaldo	Cuatro campos por canal (proveedor y modelo primario; proveedor y modelo de respaldo), con reintento y breaker de 10 min. El respaldo de un modelo local debe ser de nube. Al 24-sep-2026: 19 canales — 11 con primario local y respaldo Haiku, 6 con primario Haiku y 2 con Sonnet, ambos con respaldo local.
Criterio de asignación	Medición, no preferencia: sobre 624 generaciones. En preguntas de educación en simulación, el modelo local inventó citas en 16 de 20 respuestas, así que esos canales llevan primario en nube.
Contenido multimodal	Imágenes pasan como <code>image_url</code> ; el PDF se rasteriza (hasta 20 páginas, 150 dpi) para modelos sin PDF nativo; el audio se transcribe en el gateway por defecto (política por llave: <code>transcribe</code> o <code>native</code>); el texto plano se empalma (tope de 200 000 caracteres).
Muestreo y razonamiento	Perfil de muestreo frío automático cuando el turno trae herramientas; el razonamiento opcional se reenvía solo a motores locales.
Costo del gateway	~10 % en un solo usuario: 107 tok/s directo vs 97 tok/s por el gateway.
Tráfico real	30 días al 28-jul-2026: 5015 peticiones al motor local, 15 a la nube y 11 al servidor MLX de Apple (~99.5 % local).
Plano de control aparte	El gateway vive en otra máquina distinta a la GPU: tres reinicios del vLLM en una noche pasaron sin impacto para la flota.

6 • Arenas realizadas

Las comparaciones directas entre modelos que respaldan las decisiones de este artículo. Las cifras son las registradas en cada una; los jueces y las rúbricas se indican en cada fila.

Fecha	Qué se comparó	Criterio	Decisión y cifras
12-mar-2026	Herramienta propia para comparar 2–4 modelos de Ollama lado a lado.	23 prompts en 6 categorías; TTFT, tokens/s, tokens de salida, tiempo total, % del modelo en GPU y tiempo de carga.	Base de las arenas posteriores; sin decisión asociada.
26-mar a 5-abr-2026	Oncología (QA clínico): modelos locales y de nube, con y sin RAG, con y sin fine-tuning, en rondas de 5, 6 y 8 configuraciones.	15 casos clínicos con respuesta de referencia; juez ciego (Claude Opus); 4 criterios: precisión diagnóstica, apego a guías, completitud y utilidad clínica.	No pasaron a evaluación humana mientras las configuraciones locales no fueran competitivas; la recuperación se revisa antes de reactivarla (ver el estudio preregistrado enlazado arriba). Aquí se describe el montaje, no resultados.
28-mar-2026	Tareas de agentes en modelos locales (MLX). Pesados: Nemotron-3-Nano-30B, Mistral-Small-3.2-24B, Qwen2.5-Coder-32B, gpt-oss-20b. Ligeros: Qwen3-VL-8B, Phi-4 4B.	Puntaje por ronda (11 y 6 rondas en pesados, 7 de texto + 3 de visión en ligeros) y tokens/s.	Nemotron-3-Nano-30B: 37/55 (60–85 tok/s) frente a 36/55, 35/55 y 30/55; segunda ronda 26/30, y Qwen3.5-27B (25/30) se descartó por latencia (59–258 s). Ligeros: Qwen3-VL-8B 29/35 en texto y visión perfecta, frente a Phi-4 26/35 sin visión.

Fecha	Qué se comparó	Criterio	Decisión y cifras
2-abr-2026	Gemma 4 26B MoE, Gemma 4 31B, MiniMax M2.7, Nemotron 30B, Gemma 4 E4B y gpt-oss-20b.	6 tareas multiagente, 2 rondas (12 batallas), juez LLM automático.	Promedios 4.50 / 4.25 / 3.67 / 3.33 / 2.83 / 2.50. El 31B (14 tok/s) no justificó su lentitud frente al 26B MoE; gpt-oss-20b se descartó (code review 1/5).
12-abr-2026	Siete configuraciones para agentes: Gemma 4 31B denso local, MiniMax por NIM y directo, Nemotron 30B, GLM5, Gemma 4B y Gemma 26B local.	2 rondas de 6 prompts (delegación, conversación, DevOps, logs, code review, estrategia); juez Claude Sonnet, escala 1–5; latencia.	Gemma 31B 4.83 (~51 s), MiniMax por NIM 4.17, MiniMax directo 4.00, Nemotron 3.83, GLM5 3.50, Gemma 4B 3.50 (~16 s), Gemma 26B local 3.17. Gemma 31B quedó como premium local, Gemma 4B como respuesta rápida y la 26B local se eliminó.
11-ago-2026	Tool-use: Qwen3.6-35B-A3B (MoE, producción) frente a Muse Glimmer 30B (denso).	5 escenarios con cadenas de herramientas de mis agentes y una trampa deliberada (herramienta destructiva o prohibida); juez ciego Claude Opus; rúbrica de 4 dimensiones: herramientas, evitó la trampa, argumentos y español.	Calidad 60 vs 82/100; latencia por turno 2.2 s vs 71 s (~130 vs ~1750 tokens por turno; 80 vs 33 tok/s). El de producción ejecutó un borrado de expediente en 6 de 10 repeticiones (el otro, 0 de 10). Glimmer se apagó; los turnos sensibles o irreversibles escalan a un modelo de nube y se añaden compuertas de confirmación en código.
18-sep-2026	Ternary-Bonsai-2-27B (pesos ternarios, 1.72 bits)	75 generaciones, jueces ciegos: herramientas	Herramientas 15.93 / 16.13 / 13.13; trampas 5/15, 3/15, 6/15;

Fecha	Qué se comparó	Criterio	Decisión y cifras
	frente a Qwen3.8-27B NVFP4 y Qwen3.6-35B-A3B.	(/20), trampas, clínica (/20) y tok/s.	clínica 15.60 / 16.80 / 16.60; 32.3 / 11.1 / 49.8 tok/s. Se queda apagado: la cuantización agresiva cuesta varianza (0/3 vs 2/3 en el caso del expediente, con un 20/20 perfecto en la tercera) y calibración (una cifra inventada con tono tranquilizador). El caso de publicar sin aprobación falló 9 de 9.
28-sep-2026	¿Este turno puede disparar una acción irreversible? Un clasificador comercial de modelo pequeño, un codificador local (GLiFormer) y Qwen3.6 en producción.	120 turnos en español de México (54 irreversibles, 66 inocuos; 60 órdenes, 40 pares inocuos parecidos, 20 de negación o condicional); exactitud, falsos negativos, Brier y latencia p50.	Exactitud 98.3 / 98.3 / 82.5 %; falsos negativos 1 / 1 / 16; Brier 0.023 / 0.017 / 0.170; p50 307 / 148 / 30 ms. La calibración decidió, no la exactitud. Segunda vuelta: Qwen con <code>logprobs</code> (<code>max_tokens=1</code> , temperatura 0): 98.3 %, Brier 0.021, p50 91 ms; con umbral 0.3 cubre 92 % con 99.1 % de aciertos y 0 falsos negativos. En producción desde el 28-sep con umbral 0.8: escala a un modelo de nube.

El cómputo de IA local ya no es exótico. Por el precio de una laptop premium tengo una workstation que sirve un modelo de 35B a mis

agentes, segmenta tomografías y alimenta la búsqueda, **sin pagar por token y sin que los datos salgan de la infraestructura propia**. Y lo más honesto: lo difícil no fue el dinero ni el modelo. Fueron tres decisiones de configuración —una imagen actualizada, un parámetro de lote y un plan de recuperación ante apagones— que no venían en ninguna guía. Lo que viene: en esta misma máquina se construirá y correrá la capa de recuperación (RAG) de mi estudio de oncología preregistrado, «**Clinically Structured Retrieval: Can a Well-Designed RAG Layer Add Value Over a Base Language Model in Clinical Oncology?**».

¿Estás montando algo parecido —en tu clínica, tu centro o tu consultorio— y quieres comparar notas? **Escíbeme →**